

ML & DL

27-01-2026

Lab - 9

Step 1: Import Libraries

This step imports all necessary libraries for data processing, visualization, and machine learning.

```
In [1]: import pandas as pd
```

```
In [2]: import matplotlib.pyplot as plt
```

Step 2: Load the Dataset

Load Given dataset - heart.csv

```
In [3]: df = pd.read_csv("heart.csv")
```

Step 3: Data Overview

In this step, we examine the dataset structure, summary statistics, and check for missing values.

```
In [ ]: df.info()
```

```
In [ ]: df.describe()
```

```
In [ ]: df.isnull().sum()
```

```
In [ ]: df.shape
```

Step 4: Univariate Analysis

Here we visualize the distribution of each feature using histograms.

```
In [ ]: df.hist(figsize=(12,12))
```

Step 5: Bivariate Analysis

This step involves exploring the correlations between features using a heatmap.

```
In [9]: import seaborn as sns
```

```
In [10]: %matplotlib inline
```

```
In [ ]: sns.heatmap(df.corr(),annot=True)
```

Step 6: Outlier Detection

We visualize potential outliers using boxplots.

```
In [ ]: df.boxplot()
```

Step 7: Split Data into Training and Testing Sets

The dataset is split into training and testing sets for model evaluation.

```
In [13]: x = df.iloc[:, :-1:]
```

```
In [14]: y = df['target']
```

```
In [15]: from sklearn.model_selection import train_test_split
```

```
In [16]: x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.3,random_state=80)
```

Step 8: Train Decision Tree

We train a Decision Tree Classifier on the training data. You have to also check for KNeighborsClassifier and GaussianNB

```
In [17]: from sklearn.tree import DecisionTreeClassifier
```

```
In [18]: model = DecisionTreeClassifier()
```

```
In [ ]: model.fit(x_train,y_train)
```

```
In [20]: from sklearn.metrics import accuracy_score
```

```
In [21]: y_predict = model.predict(x_test)
```

```
In [ ]: accuracy_score(y_test,y_predict)
```

```
In [ ]: model.score(x_test,y_test)
```

Cross Validation

```
In [24]: from sklearn.model_selection import KFold
```

```
In [25]: k = KFold(n_splits=10)
```

```
In [ ]: x
```

```
In [ ]: for train_index,test_index in k.split(x):  
        x_train,x_test = x.values[train_index],x.values[test_index]  
        y_train,y_test = y[train_index],y[test_index]  
        model1 = DecisionTreeClassifier()  
        model1.fit(x_train,y_train)  
        print(model1.score(x_test,y_test))
```

```
In [28]: from sklearn.model_selection import cross_val_score
```

```
In [ ]: cross_val_score(model,x,y,cv=10)
```

```
In [30]: import numpy as np
```

```
In [31]: x1 = np.arange(1,11).reshape(-1,1)
```

```
In [32]: y1 = np.array([0,1,0,1,0,1,0,1,0,1])
```

```
In [ ]: for i,j in k.split(x1):  
        print("<----->")  
        print(x1[i])  
        print(x1[j])  
        x_train,x_test = x1[i],x1[j]
```

```
y_train,y_test = y1[i],y1[j]
```

Step 11: Train Bagging Classifier

We train a Bagging Classifier with Decision Trees as the base model.

```
In [34]: from sklearn.ensemble import BaggingClassifier
```

```
In [35]: bc = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=5)
```

```
In [ ]: bc.fit(x_train, y_train)
```

```
In [37]: y_predict = bc.predict(x_test)
```

```
In [ ]: accuracy_score(y_test, y_predict)
```

Step 12: Evaluate Bagging Classifier

The Bagging model is evaluated using accuracy.

```
In [ ]: from sklearn.metrics import accuracy_score

y_pred_bag = bc.predict(x_test)

bag_accuracy = accuracy_score(y_test, y_pred_bag)
print("Bagging Accuracy:", bag_accuracy)
```

Step 13: Train Random Forest

We train a Random Forest Classifier on the dataset.

```
In [39]: from sklearn.ensemble import RandomForestClassifier
```

```
In [40]: rfc = RandomForestClassifier(n_estimators=100, max_features='sqrt')
```

```
In [ ]: rfc.fit(x_train, y_train)
```

```
In [42]: y_predict = rfc.predict(x_test)
```

```
In [ ]: accuracy_score(y_test, y_predict)
```

Step 14: Feature Importance in Random Forest

We analyze feature importance as determined by the Random Forest model.

```
In [ ]: rfc.feature_importances_
```

```
In [ ]: import sklearn
print(sklearn.__version__)
```

```
In [ ]: rfc.feature_names_in_
```

Step 15: Evaluate Random Forest

We evaluate the Random Forest model using accuracy.

```
In [ ]: y_pred_rf = rfc.predict(x_test)

rf_accuracy = accuracy_score(y_test, y_pred_rf)
print("Random Forest Accuracy:", rf_accuracy)
```

Step 16: Train AdaBoost Classifier

We train an AdaBoost Classifier on the dataset.

```
In [ ]: from sklearn.ensemble import AdaBoostClassifier
```

```
In [ ]: ada_model = AdaBoostClassifier(n_estimators=100, random_state=42)
```

```
In [ ]: ada_model.fit(x_train, y_train)
```

Step 17: Evaluate AdaBoost Classifier

The AdaBoost model is evaluated using accuracy.

```
In [ ]: y_pred_ada = ada_model.predict(x_test)
```

```
In [ ]: ada_accuracy = accuracy_score(y_test, y_pred_ada)
```

```
In [ ]: print("AdaBoost Accuracy:", ada_accuracy)
```

<---- B ---->

Cost Complexity Pruning for Decision Tree

```
In [ ]: clf = DecisionTreeClassifier(random_state=42)
path = clf.cost_complexity_pruning_path(x_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
```

```
In [ ]: clfs = []
for ccp_alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=42, ccp_alpha=ccp_alpha)
    clf.fit(x_train, y_train)
    clfs.append(clf)
```

```
In [ ]: clfs = clfs[:-1]
ccp_alphas = ccp_alphas[:-1]
```

Random Forest with Iterative Training (OOB Error)

```
In [ ]: rf = RandomForestClassifier(oob_score=True, warm_start=True, random_state=42)

oob_error_rates = []
n_estimators_list = list(range(10, 201, 10))

for n in n_estimators_list:
    rf.set_params(n_estimators=n)
    rf.fit(x_train, y_train)

    # OOB Error = 1 - OOB Score
    oob_error = 1 - rf.oob_score_
    oob_error_rates.append(oob_error)
```

Receiver Operating Characteristic (ROC) Analysis

```
In [ ]: from sklearn.metrics import roc_curve, auc

y_prob = rf.predict_proba(x_test)[: , 1]

# Calculate ROC curve metrics
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

# Plot ROC Curve
```